Legal Saathi - solution document

Problem 2 - everyday legal problems for people who cannot afford a lawyer. Flask + Python, swappable OpenAI-compatible model. Source, tests and raw outputs ship with this document.

1. Problem and why

A tenant's deposit is withheld, or an online refund never arrives. The amount (Rs 20,000-2,00,000) is too small to justify a lawyer, but the person does not know which forum applies, what the deadline is, or what to do this week. The useful answer is not "what is the law" but "where does this go, how long do I have, what do I do next".

Both domains were chosen because they are frequent, the amounts are small, and each has a known forum and a known time limit - so useful direction is possible without guessing. A family or criminal matter has no such single correct next step, and a wrong answer there is unacceptable.

Both also hide one clause that misleads people: "disputes shall be referred to a sole arbitrator appointed by the Company / Landlord". Readers assume they have signed away their rights. So the app detects that clause, quotes it, and explains it in plain words.

2. Assumptions

- Audience: urban smartphone user, English / Hindi / Hinglish, no lawyer.
- Scope: tenant security deposit and consumer refund only. Out-of-scope messages are refused kindly and pointed to a lawyer or the District Legal Services Authority.
- All test documents and conversations are synthetic. No real personal data.
- The 15 knowledge-base entries are unverified (all `verified: false`); a lawyer must check each against the official source before real use.
- No OCR: scanned PDFs and photos are rejected with a clear message.
- One in-memory session, 2-hour idle expiry. No login, no database, no history.
- The model is an untrusted component. Code owns every safety decision.

3. How it works

Request flow: Browser -> Flask route -> service -> model. The service composes:

- `prompts.py` + `schemas.py` - what to ask, and the JSON shape expected back
- `kb.json` / `kb.py` - the only source of legal text
- `llm.py` - Groq, Gemini or any OpenAI-compatible endpoint
- `checks.py` - verify quotes, remove unverified law, decide escalation
- `docs.py` / `store.py` - extract and chunk in memory, hold the session

Task	Done by	Why
Understand a messy story, extract facts	Model	Language understanding
Supply the actual rule text	Knowledge base only	Prevents invented law
Parse and order dates	Code (`dateutil`)	Deterministic
Prove a quote exists in the source	Code	A model cannot verify itself
Delete unverified legal references	Code (regex vs allowlist)	Safety net
Decide "see a lawyer"	Code rules	Auditable and testable
Keep the reply to one question	Code	A prompt alone was not reliable
Merge chunks, de-duplicate, sort	Code	Cheap and deterministic
Wording, plain summaries, notice drafting	Model	Writing quality

Swapping models is a `.env` edit only - `LLM_PROVIDER_ORDER`, `<NAME>_API_KEY`, `<NAME>_BASE_URL`, `<NAME>_MODEL`. `llm.py` is the only module that imports `openai`, so no provider-specific code exists anywhere else.

4. Why it cannot invent law

1. **Knowledge base only.** Every legal statement must carry a `[KB:id]` tag. The model is shown only rule text from `kb.json`; it has no other legal input.
2. **Citation scan.** Code removes any sentence citing a section, Act, case name or penal code that is not in the KB allowlist, then tells the user a part was removed. Unknown tags are stripped.
3. **Quote verification.** Every amount, date, clause and timeline event must quote the source verbatim. Code checks the string against the document and drops whatever it cannot find, reporting `dropped_unverified`.
4. **Code decides escalation.** `none / advise / urgent` comes from keyword, value, date and time-bar rules in Python. The model may raise the level, never lower it. When the KB has nothing relevant, the prompt requires "I am not sure" plus a pointer to a lawyer.

5. Scale and cost

Measured on one full case (1 chat turn + summary + timeline + notice, rental agreement): **6,935 prompt + 2,422 completion tokens**, about 5 s of model time, with occasional 429 back-off on the free tier.

Cost per case = (prompt_tokens x price_in + completion_tokens x price_out) / 1e6. With an illustrative small-model price of $0.10 / $0.30 per 1M tokens that is about **$0.0014 per case, or $1.40 per 1,000 cases**. A chat-only session is far cheaper (about 3,000 tokens). The demo runs inside free tiers.

Adding a state or topic means adding KB entries and tests - no code change. Human-in-the-loop today: a lawyer reviews `kb.json` before `verified` is set, and a notice is a draft the user must review. Monitoring: every response carries flags (`unverified_reference_removed`, `unverified_number`, `possible_prompt_injection`, `partial_result`) and a `dropped_unverified` count, and each task is archived locally under `jobs/<date>/<job_id>/` for inspection.

6. Results

- **143 unit tests pass offline** (no key): citation scan, quote verification, date parsing, escalation monotonicity, injection, upload validation, every endpoint and every error code.
- **12 evaluation cases** play chat, upload, summary, timeline and notice end to end; raw outputs are kept unedited in `outputs/run_20261108_1708/`.

Metric	Value
Rules passed	85 / 97
Domain routing	7 / 12
Single-question rule	12 / 12
Over-escalation	0
Arbitration detection	3 / 4

These numbers come from the **offline provider**, which always answers as a deposit case; the refund cases fail domain by construction. They prove the harness and the checks, not model quality. A real Groq run is one command: `uv run python scripts/run_eval.py provider groq`.

Three honest failures:

1. **Case names survived the citation scan.** "Sharma v. Verma" was split at "v." into two fragments, so the case-name pattern never matched. Caught by reading regex output on an ordinary sentence. Fix: the sentence splitter keeps abbreviations attached.
2. **A retired model id looked like a broken app.** Every chat returned the fallback message; the log showed HTTP 404. The provider had retired `llama-3.3-70b-versatile`. Worse, the code retried that 404 three times with back-off (8 s per turn) and discarded the status code. Fix: 4xx other than 429 is non-retryable, and the provider's status and error code are logged.
3. **The UI printed raw markdown.** Replies contained `1. **Send a Legal Notice:` **and the page showed the asterisks, because model output was written** as plain text. Caught in a browser snapshot after the user said it "felt like a traditional chatbot". Fix: a small DOM-only renderer for bold, italics, lists and chips.

7. Tools and where they were wrong

Tools: an AI coding assistant in VS Code for implementation and tests; Groq (`qwen/qwen3.8-27b`, chosen because it supports native JSON mode and needs roughly a third of the tokens of the gpt-oss models, which reject `json_object`); `uv`; `pytest`.

The tool's failures were real and are logged in full in `documentation/08`. Beyond the three above: an escalation keyword copied literally ("lock changed") missed "changed the lock"; `Rs. 95,000` was never number-checked because of a regex word boundary; and a Title-Case notice placeholder (`[Your Name]`) was invisible to an uppercase-only extractor, so 1 of 5 blanks was reported. Each was found by running the code, not by reading it, which is why the test suite exists.

8. Limits and next steps

Does not cover: criminal, eviction, cheque bounce, family, property or tax matters; scanned documents; voice; court filing; multiple users.

Next, in order: have a lawyer verify the 15 KB entries and fill `source_url`; add state-level deposit entries; validate case-file facts against the user's own words so a wrong amount cannot enter silently; stream replies only once each chunk can be validated; and add a lawyer-reviewed notice path so the draft becomes something a person can actually send. Outcome prediction and automatic court filing are deliberately not built.